SentinelSOC

Mini Security Operations Center

Detect . Investigate . Defend

PROJECT TYPE: Cybersecurity Web Application

PURPOSE: IBM Cybersecurity Bootcamp -- Final Project

TECHNOLOGY: Python 3.14 . Flask 3.1 . SQLite . Chart.js

VERSION: 1.0

GENERATED: October 21, 2026 19:55

1	47	5	9	3
APP FILE	TESTS PASS	DET. RULES	PAGES	USER ROLES

[!] DEMO DATA -- Educational Prototype -- Not for Production Use

1. Project Overview

SentinelSOC (Mini Security Operations Center) is a full-stack defensive cybersecurity web application built for the IBM Cybersecurity Bootcamp. It ingests security log files, detects suspicious authentication patterns using five rule-based detection algorithms, generates explainable alerts, manages security incidents, and presents all findings in a professional dashboard with light and dark mode support.

2. Problem Statement

Security Operations Center analysts review thousands of log events every day. Manually identifying attack patterns -- such as brute-force login attempts or password spraying -- from raw log data is slow, error-prone, and exhausting.

SentinelSOC demonstrates how a rule-based detection engine can automatically surface suspicious patterns so analysts can focus on investigation and response rather than log triage. Every alert includes a plain-English explanation and evidence so the analyst always understands WHY the alert fired.

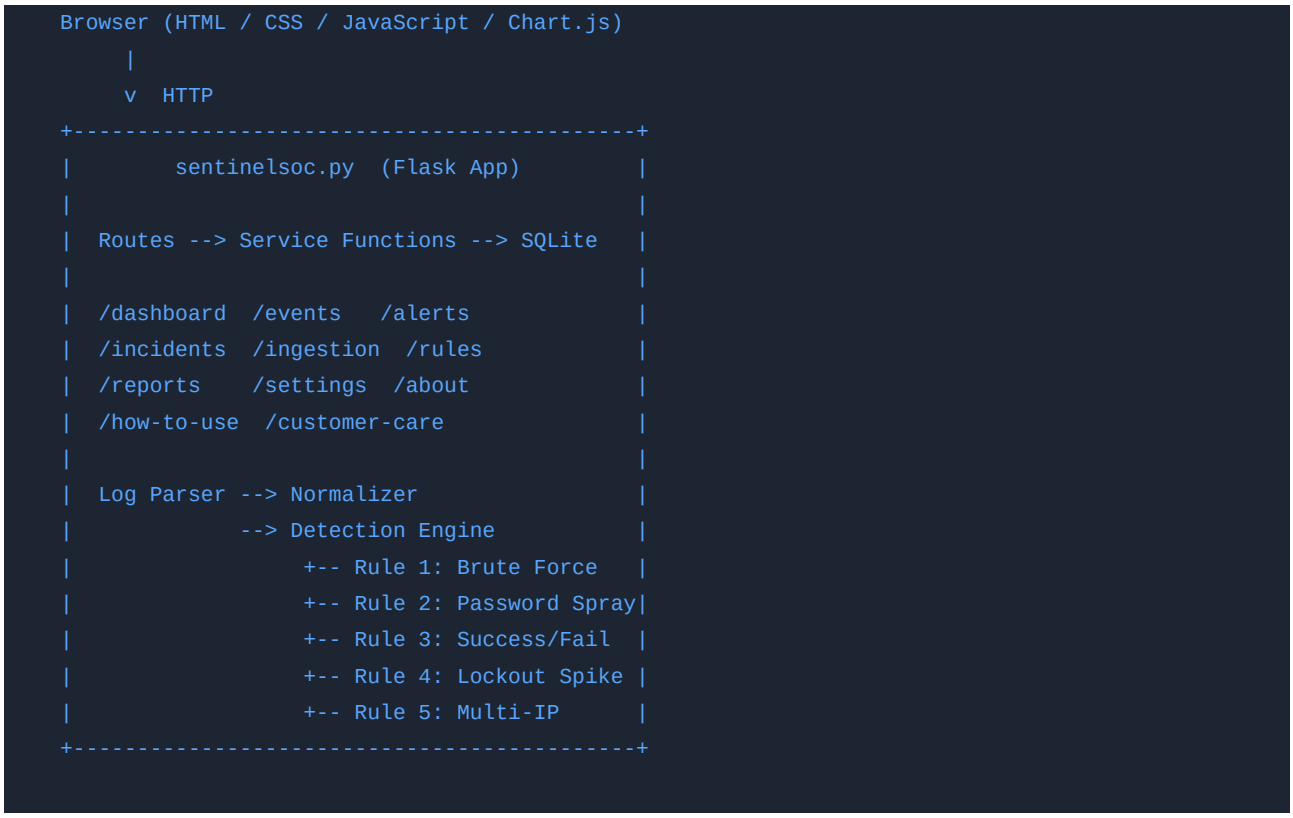
3. Objectives

1. Process CSV and JSON Lines security log files uploaded by analysts.
2. Normalize raw log fields (timestamps, IPs, auth results) into a consistent format.
3. Detect five categories of suspicious authentication behaviour automatically.
4. Generate explainable alerts with full evidence and investigation recommendations.
5. Allow analysts to manage alert status and escalate to incidents.
6. Track incident investigations with notes, linked alerts, and resolution records.
7. Export events, alerts, and reports as CSV for further analysis.
8. Provide a professional responsive security dashboard (dark + light mode).
9. Protect the application with login auth, input validation, and security headers.
10. Pass a full automated test suite (47 tests) covering all core features.

4. System Architecture

SentinelSOC is implemented as a single Python file using the Flask micro-framework. The architecture follows a layered design: HTTP routes delegate to service functions that operate on a SQLite database. All templates are rendered server-side using Jinja2. Chart.js handles client-side data visualisation.

Architecture Diagram



Technology Stack

Backend Language	Python 3.14
Web Framework	Flask 3.1
Database	SQLite (WAL mode, parameterized queries)
Frontend	HTML5 / CSS3 / Vanilla JavaScript
Charts	Chart.js 4
Icons	Lucide Icons (CDN)
Testing	pytest 8 + pytest-cov
Authentication	Session-based (demo credentials)
PDF Generation	fpdf2
Deployment	Local -- python sentinelsoc.py

5. Database Schema

SQLite database with 7 tables, foreign key constraints, and indexes on frequently queried columns. All queries use parameterized statements.

Table Name	Purpose & Key Columns
security_events	Parsed log events: timestamp, source_ip, username, event_type, auth_result
ingestion_batches	Each file upload: filename, accepted/rejected counts, format, ingested_at
detection_rules	Built-in rules: rule_key, name, severity, threshold, window_minutes
alerts	Generated alerts: rule_key, severity, status, evidence (JSON), dedup_key
incidents	Investigation records: title, severity, status, assigned_to, resolution
incident_alerts	Many-to-many join: links alerts to incidents
incident_notes	Free-text notes attached to incidents with author and timestamp

Security Controls

SQL Injection	All queries use parameterized statements -- no string concatenation
XSS	Jinja2 auto-escaping on all template variables
Path Traversal	os.path.basename() strips directory components from filenames
File Upload	5 MB limit; decoded safely; files never stored to disk or executed
CSV Injection	Cells starting with =+-@|% prefixed with apostrophe
Security Headers	X-Content-Type-Options, X-Frame-Options, X-XSS-Protection
Input Validation	Status and severity values checked against explicit allowlists
Authentication	Session-based login; all routes protected by before_request hook
No Auto-Remediation	App never blocks IPs, locks accounts, or runs system commands

6. Detection Rules

All five rules are configurable (threshold and time window adjustable in Settings). Each alert includes evidence referencing the underlying events and a deduplication key to prevent the same alert from being generated twice.

Rule 1 — Repeated Failed Logins (Brute Force)

HIGH — 5+ failed login attempts from the same source IP within 5 minutes.

REC: Investigate the source IP. Review affected accounts.

Rule 2 — Multiple-Account Login Failures (Password Spray)

HIGH — 1 source IP failing against 3+ distinct usernames within 10 minutes.

REC: Review all targeted accounts. Investigate the source IP.

Rule 3 — Successful Login After Repeated Failures

HIGH — 3+ failures for same username+IP followed by a success within 10 minutes.

REC: Contact the account owner to verify the login was legitimate.

Rule 4 — Account Lockout Spike

MEDIUM — 5+ account lockout events within 10 minutes.

REC: Unlock affected accounts. Identify source of failed attempts.

Rule 5 — Suspicious Authentication Pattern (Multi-IP)

MEDIUM — 3+ distinct source IPs failing against the same account within 15 minutes.

REC: Verify with the account owner. Preserve relevant log entries.

7. Application Features

Feature	Description
Overview Dashboard	Event/alert counts, Chart.js trend + donut + bar charts, recent tables
Security Events	Paginated list, search/filter by IP/user/result/date, CSV export
Alerts	Alert list, severity badges, detail view, status update, incident creation
Incidents	Create, link alerts, add notes, assign analyst, status, resolution
Log Ingestion	CSV + JSON Lines upload, drag-and-drop, validation summary, auto-detection
Detection Rules	View 5 rules with description, threshold, window, alert count
Reports	Aggregated metrics, trend charts, top rules/IPs, printable report, CSV exports
Settings	Per-rule threshold/window config, manual detection run, demo data reset
About Website	Platform info, user roles, tech stack, important notices
How to Use	8-step workflow guide, log format reference, alert status guide, tips
Customer Care	Contact cards (email/chat/phone), FAQ accordion, contact form
Dark / Light Mode	Toggle button in topbar; choice saved in localStorage; no page flash
Login / Logout	3 demo accounts (admin/analyst/viewer); all routes protected

8. Test Results

Automated test suite using pytest. All 47 tests pass on Python 3.14 -- 0 failures, 0 warnings.

Test Category	Count	Coverage
CSV Parsing	5	Valid, missing cols, empty, header-only, extra cols
JSON Lines Parsing	3	Valid, invalid lines, blank lines skipped
Log Normalizer	7	Valid record, bad ts, missing type, bad IP, auth mapping
Brute Force Rule	4	Threshold, below threshold, window boundary, dedup
Password Spray Rule	2	Multi-account trigger, same account (no spray)
Success After Fail	2	Trigger on success after fails, no trigger on solo success
Lockout Spike Rule	2	Spike trigger, below threshold
Multi-IP Rule	1	Multiple IPs targeting same account
Empty Database	4	Detection, dashboard, alerts page, incidents page
Alert Status	3	Update, invalid rejected, all valid statuses
Incident Management	5	Create, empty title, note, link alert, invalid ID
File Upload	4	Valid CSV, valid JSONL, no file, invalid records
CSV Export Safety	3	Formula injection, normal values, None handling
DB Persistence	2	Events stored, alerts across connections
TOTAL	**47**	All passing -- 0 failures, 0 warnings

9. Demo Scenarios (Pre-loaded)

On first launch SentinelSOC seeds 110 synthetic events using RFC 5737 documentation IP ranges (198.51.100.x, 203.0.113.x) and fictional usernames. All 5 rules are triggered.

Scenario	What Happens	Result
Brute Force	6 failures from 198.51.100.42 against alice.smith in < 5 min	High Alert
Password Spray	1 IP targeting 4 accounts (alice/bob/charlie/diana) in < 10 min	High Alert
Success/Fail	4 failures then a success for bob.jones from attacker IP	High Alert
Lockout Spike	7 account lockouts across 7 users within 10 minutes	Medium Alert
Multi-IP Attack	4 distinct IPs (203.0.113.10-13) targeting charlie.dev < 15 min	Medium Alert

10. User Accounts

Username	Password	Role	Access Level
`admin`	`Admin@1234`	Administrator	Full access -- settings, reset, configure rules
`analyst`	`Analyst@1234`	SOC Analyst	Standard -- investigate alerts, manage incidents
`viewer`	`Viewer@1234`	Read-Only	View dashboard, events, alerts, reports

11. Project Files

File	Purpose
`sentinelsoc.py`	Complete app -- routes, DB, detection engine, templates
`requirements.txt`	Python dependencies (Flask, pytest, fpdf2)
`generate_report.py`	This PDF generator
`README.md`	Full setup guide, architecture, demo walkthrough
`.env.example`	Environment variable template
`tests/test_sentinelsoc.py`	47 automated pytest tests
`sentinelsoc.db`	SQLite database (auto-created, not committed to git)
`SentinelSOC_Project_Report.pdf`	This report

12. Quick Start (Windows)

Step 1	**Open Command Prompt**	`Press Win + R, type cmd, press Enter`
Step 2	**Go to project folder**	`cd /d E:\bootcamp-project`
Step 3	**Run the application**	`python sentinelsoc.py`
Step 4	**Open in browser**	`http://127.0.0.1:5000  (admin / Admin@1234)`
Step 5	**Run tests (optional)**	`python -m pytest tests/ -v`
Step 6	**Stop the app**	`Press Ctrl + C in the terminal`

13. Limitations

1. Demo accounts only -- not suitable for public deployment without proper user management.
2. No TLS -- runs on HTTP; add a reverse proxy with HTTPS for any production-like use.
3. Synthetic data only -- do not process real user credentials without authorisation.
4. Rule-based detection only -- novel attack patterns not matching 5 rules go undetected.
5. No live monitoring -- detection runs on upload or manual trigger, not as a stream.
6. Detections are indicators -- all alerts require analyst investigation before action.

14. Responsible Use Statement

SentinelSOC is built strictly for educational and demonstration purposes as part of the IBM Cybersecurity Bootcamp.

* Do NOT use this tool to monitor real users without explicit knowledge and consent.
* Do NOT process real authentication credentials, passwords, or personal data.
* Do NOT automatically block IP addresses or lock accounts based solely on these alerts.
* Always follow your organisation's security policies and applicable laws.
* This prototype is NOT production-ready. Deploy only on a trusted local machine.

Every alert is an INDICATOR of suspicious behaviour requiring human investigation. The application never claims a confirmed attack without analyst review.

15. Sign-off

Project Name:	SentinelSOC -- Mini Security Operations Center
Tagline:	Detect. Investigate. Defend.
Bootcamp:	IBM Cybersecurity Bootcamp
Developer:	**Daksh.A.Saraswala**
Email:	**Dakshsaraswala23@gmail.com**
Contact:	**+91 6359058845**

College:	**Dr.S.&S.S.Gandhi College Of Engineering & Technology**
Program:	**Diploma College**
App File:	sentinelsoc.py (single-file Flask application)
Test Suite:	47 / 47 tests passing -- pytest 8 on Python 3.14
Report Date:	October 21, 2026

Built with Python - Flask - SQLite - Chart.js - fpdf2